

A PROJECT REPORT ON

SMART BUS TRACKING AND CROWD PREDICTION FOR PUBLIC TRANSPORT

Domain: Transport / IoT + ML

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 IoT Hardware Components	
	4.3 Software Specification	
V	SYSTEM ANALYSIS	10

	5.1 Feasibility Study	
	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Public buses are the lifeline of daily travel for students, workers and villagers in Tamil Nadu and across India. Town and mofussil buses carry millions of passengers every day. Yet passengers usually wait at bus stops without knowing when the next bus will arrive, whether it has already passed, or whether it will be too crowded to board.

On the same route, one bus may be overcrowded while the next one is nearly empty, because buses bunch together and schedules are not adjusted to demand. Operators have little data on actual running times, delays and passenger load at different times of the day, so they cannot plan schedules and frequency effectively.

GPS tracking and machine learning can address these problems. This project, BusNow, fits a low-cost GPS device (ESP32 with NEO-6M GPS) on each bus to report its location every few seconds. The system matches the location to route stops, predicts the arrival time (ETA) at upcoming stops using historical speeds and live traffic, and predicts how crowded the next bus will be using ticket counts and a regression model. Passengers see live buses with ETA and crowd level on a map, and operators view delays and demand to adjust schedules.

LIMITATIONS

1. Passengers wait without knowing when a bus will arrive.
2. Overcrowded and empty buses run on the same route.
3. Operators lack data for scheduling.
4. Rural routes have no real-time information at all.

1.1 History of Domain

Automatic Vehicle Location (AVL) systems for buses began in the 1970s and 1980s using signpost and radio technologies. With the GPS system becoming fully available for civilian use in the 1990s, transit agencies adopted GPS-based AVL, and the

General Transit Feed Specification (GTFS, 2005) and GTFS-Realtime (2011) standardised schedule and live data exchange.

In India, cities such as Bengaluru (BMTCL), Delhi and Chennai (MTC) introduced GPS tracking and passenger information apps. Research on bus arrival time prediction has evolved from historical averages and Kalman filters to machine learning models such as SVR, gradient boosting and LSTM, while crowding prediction uses ticketing and automatic passenger counter data.

1.2 Overview of the Project

Each bus has an ESP32 with a NEO-6M GPS module and a GSM/4G modem (or uses the conductor's phone), publishing latitude, longitude, speed and heading every 5 seconds over MQTT. The server matches each point to the route shape and determines the last passed stop and the distance to upcoming stops. Ticket counts from the electronic ticket machine (or the conductor app) give boarding and alighting by stop.

An ETA model (gradient boosting regression) predicts segment travel times using time of day, day of week, historical segment speed, current speed of the bus and recent speeds of other buses on the segment. A crowd model predicts the occupancy percentage at each upcoming stop from current load, typical boarding/alighting and time features, classified as Seats available, Standing or Full. Passengers subscribe to a stop to receive notifications when the bus is 5 minutes away. Operators get route performance reports.

1.3 Motivation

Uncertainty about bus arrival and crowding is one of the main reasons people shift to private vehicles.

- **Motivation:** To make public buses predictable and comfortable by giving accurate ETA and crowd information.
- **Outcome:** Less waiting time, better distribution of passengers, data-driven scheduling and increased public transport use.

1.4 Aim of the Project

The project is entitled as “**SMART BUS TRACKING AND CROWD PREDICTION FOR PUBLIC TRANSPORT**”. The aim of this project is to develop an IoT and machine learning based bus tracking system that shows live bus locations, predicts arrival times and crowd levels at stops, notifies passengers, and provides operators with performance and demand reports.

1.5 Objectives

- To build a low-cost GPS tracking unit using ESP32 and NEO-6M.
- To map-match bus locations to route stops.
- To predict ETA using historical and live speeds.
- To estimate and predict bus occupancy.
- To provide passenger notifications and operator reports.

1.6 Literature Review

Jeong and Rilett (2004): Compared historical average, regression and artificial neural network models for bus arrival time prediction and found that ANNs performed best using AVL data.

Kalman (1960): Introduced the Kalman filter, a recursive estimator widely used in vehicle tracking and in early bus arrival time prediction systems.

Friedman (2001): Proposed gradient boosting machines, which build additive regression models by fitting trees to residuals, forming the basis of modern boosted tree ETA models.

Google - GTFS Realtime: The GTFS-Realtime specification defines feeds for vehicle positions, trip updates and service alerts, enabling interoperability of transit data with journey planners.

Jenelius (2019): Studied data-driven metro train crowding prediction based on real-time load data and showed that crowding can be predicted accurately several stations ahead.

Ministry of Housing and Urban Affairs: Smart Cities Mission projects promote Intelligent Transport Management Systems including vehicle tracking and passenger information systems for city buses.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, passengers rely on printed timetables (often outdated) or on asking others. Some city buses have GPS tracking, but rural and town buses usually do not. Crowding is known only when the bus arrives. Depots record running times manually.

DISADVANTAGES

- Long, uncertain waiting times.
- No crowd information before boarding.
- Bus bunching not detected.
- Manual records unsuitable for analysis.
- Expensive commercial AVL systems.

2.2 Proposed System

The proposed BusNow system uses a low-cost ESP32 GPS unit on each bus and MQTT to send live locations. Machine learning models predict ETA and crowd levels at each stop. A Leaflet map shows live buses; passengers get alerts; and operators see delays, bunching and demand reports to adjust schedules.

ADVANTAGES

- Accurate live ETA at every stop.
- Crowd level prediction before the bus arrives.
- Low-cost hardware suitable for rural buses.
- Bunching and delay detection for operators.
- Data for evidence-based scheduling.

2.3 Project Schedule

The development of BusNow follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the BusNow system are listed below.

- **GPS reporting:** Bus unit sends location, speed and heading every 5 seconds.
- **Map matching:** Match location to route shape and stops.
- **ETA prediction:** Predict arrival times at upcoming stops.
- **Occupancy estimation:** Compute load from ticket counts.
- **Crowd prediction:** Predict crowd level at upcoming stops.
- **Live map:** Show buses with ETA and crowd level.
- **Notifications:** Notify subscribed passengers before arrival.
- **Reports:** Route punctuality, delays and demand reports.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Accuracy:** ETA error within 2 minutes for 80% of predictions up to 20 minutes ahead.
- **Latency:** Live position on map within 10 seconds.

- **Reliability:** Device buffers data during network loss.
- **Scalability:** Support 500 buses on one server.
- **Low cost:** Tracking unit below Rs. 1,500.
- **Usability:** Simple stop search in Tamil and English.

3.3 User Requirements

- See when the next bus will come to my stop
- Know whether the bus is crowded
- Get a reminder before the bus arrives
- Operators see delays and demand
- Works on basic smartphones

3.4 System Requirements

- ESP32 + NEO-6M GPS + SIM800L/4G modem per bus
- MQTT broker
- Node.js server
- MongoDB with geospatial indexes
- Python ML service
- Leaflet web app

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above

Hard : 512 GB

Disk

RAM : 8 GB

Monitor : 15" colour monitor

Bus unit : ESP32, NEO-6M GPS, SIM800L GSM / 4G module, 12V-5V converter

4.2 IoT Hardware Components

S.No	Component	Purpose
1	ESP32 DevKit	Controller and MQTT client
2	u-blox NEO-6M GPS with antenna	Location, speed and heading
3	SIM800L / SIM7600 module	Mobile data connectivity
4	LM2596 buck converter	12/24 V bus supply to 5 V
5	Optional IR passenger counter	Door boarding/alighting counts

4.3 Software Specification

Operating System : Ubuntu 22.04 / Windows 10

Firmware : Arduino IDE (C++), TinyGPS++

Protocol : MQTT (Mosquitto)

Back End : Node.js with Express

ML : Python, scikit-learn / LightGBM

Database : MongoDB

Front End : Leaflet.js, HTML/JS

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed BusNow system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

GPS modules, ESP32 and MQTT are reliable and inexpensive. Map matching and ETA prediction can be done with standard Python libraries. The project is technically feasible.

ECONOMIC FEASIBILITY

A tracking unit costs about Rs. 1,200 per bus, much lower than commercial AVL systems. Software is open source. It is economically feasible for transport corporations and private operators.

OPERATIONAL FEASIBILITY

Drivers need do nothing; the device starts with the ignition. Passengers use a simple web page. Operators get automatic reports. It is operationally feasible.

5.2 About the Software

ESP32 MICROCONTROLLER

The ESP32 is a low-cost, low-power system-on-chip microcontroller from Espressif Systems with integrated Wi-Fi and dual-mode Bluetooth. It has a dual-core Tensilica Xtensa LX6 processor running up to 240 MHz, 520 KB SRAM, and many GPIO, ADC, I2C, SPI and UART pins.

The ESP32 can be programmed using the Arduino IDE in C/C++ or with MicroPython. Its built-in Wi-Fi lets sensor readings be sent directly to a cloud server using

MQTT or HTTP without any additional module.

- Built-in Wi-Fi and Bluetooth
- 12-bit ADC for analog sensors
- Deep-sleep mode for battery operation
- Costs less than Rs. 500

NEO-6M GPS MODULE

The u-blox NEO-6M is a stand-alone GPS receiver with a sensitivity of -161 dBm and a position accuracy of about 2.5 m. It outputs standard NMEA sentences over UART at 9600 baud, which are parsed with the TinyGPS++ library on the ESP32.

MQTT PROTOCOL

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency or unreliable networks. It is an OASIS standard and is widely used in IoT.

Devices publish messages to topics on a broker (such as Mosquitto or HiveMQ), and any application subscribed to that topic receives the message instantly. MQTT supports three Quality of Service levels and retained messages.

- Very small packet overhead
- Publish-subscribe decoupling
- Works well on poor mobile networks

NODE.JS AND EXPRESS

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive real-time applications.

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building REST APIs, including routing, middleware and

request/response handling. Together they allow both front end and back end to be written in JavaScript.

- Handles many concurrent connections
- npm - the largest package registry
- Ideal for real-time and IoT back ends

MONGODB

MongoDB is a source-available, cross-platform, document-oriented NoSQL database. It stores data as flexible JSON-like documents (BSON), which means fields can vary from document to document and the data structure can be changed over time.

MongoDB supports indexing, aggregation pipelines, geospatial queries and time-series collections, which makes it well suited for storing sensor readings, user profiles and application logs.

- Schema-less flexible documents
- Horizontal scaling through sharding
- Powerful aggregation framework

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain

- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

LEAFLET.JS

Leaflet is the leading open-source JavaScript library for mobile-friendly interactive maps. It is lightweight (about 42 KB) and has all the mapping features most developers need, such as tile layers, markers, pop-ups, polylines and GeoJSON layers.

Leaflet works with free OpenStreetMap tiles, so maps can be shown without paying for a commercial map API.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of BusNow is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

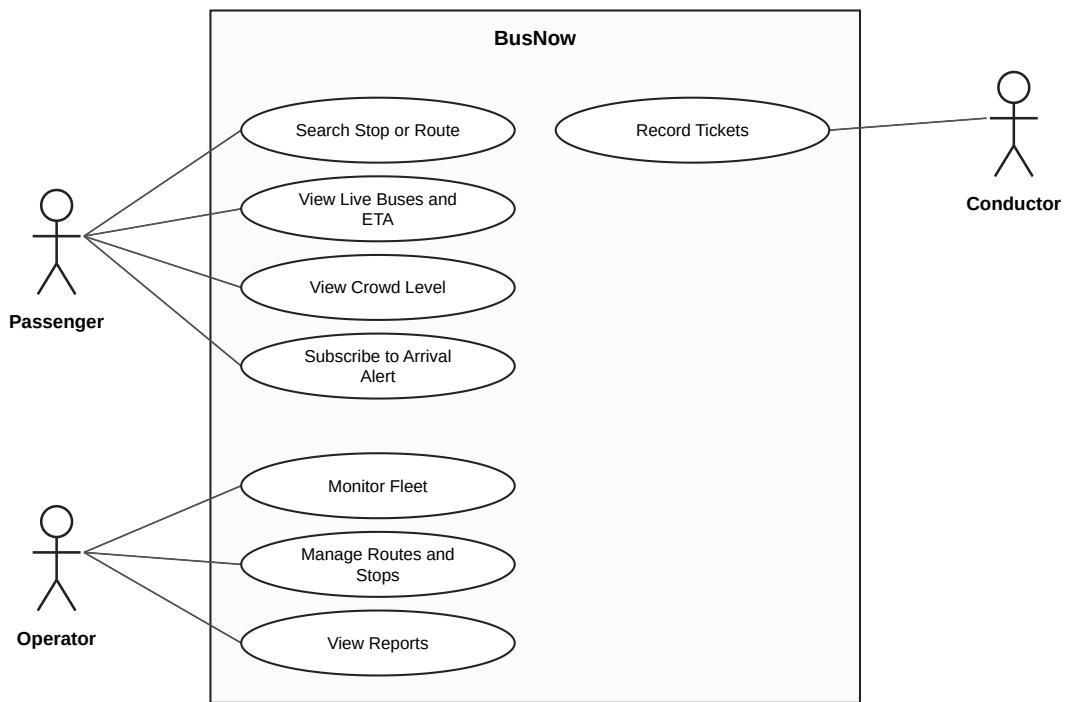


Fig 1: Use case diagram of BusNow

6.2 System Architecture

BusNow uses a layered IoT architecture with devices on buses, an MQTT messaging layer, a processing layer with map matching and ML services, and an application layer for passengers and operators.

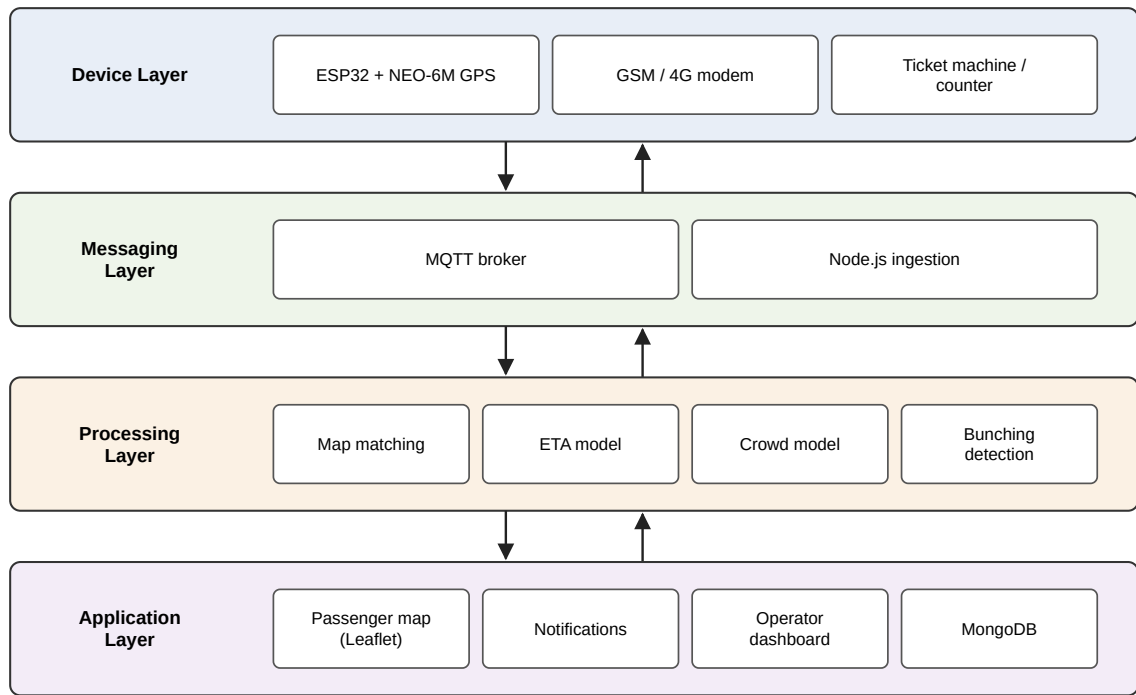


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

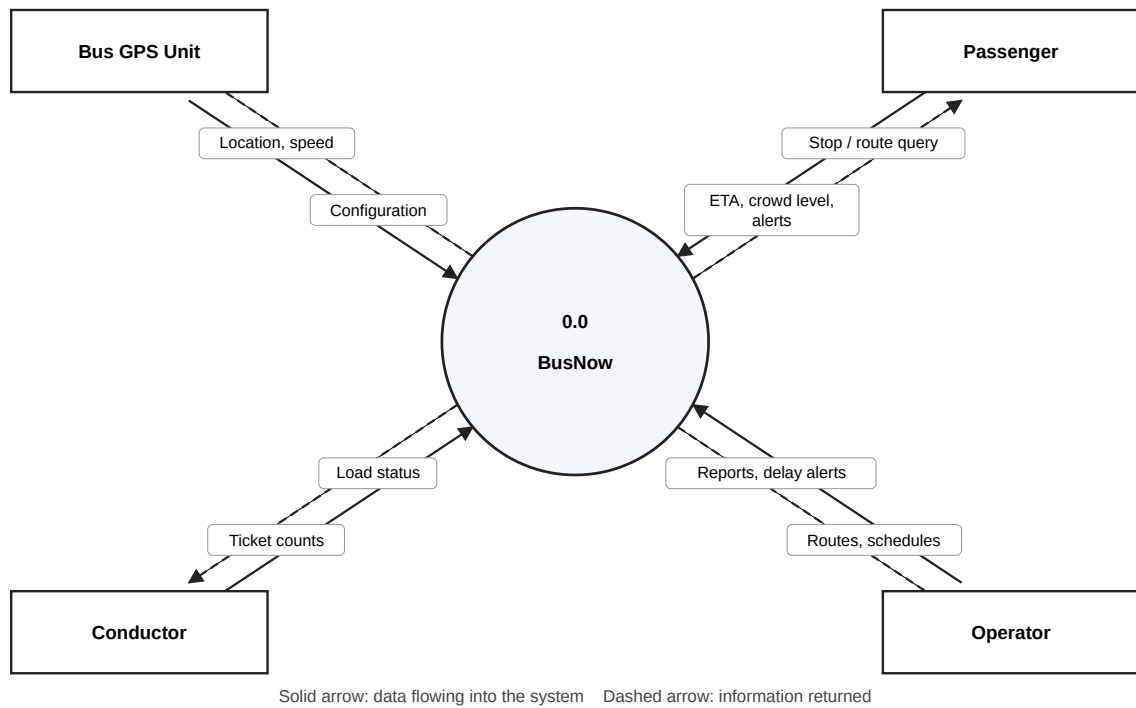


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

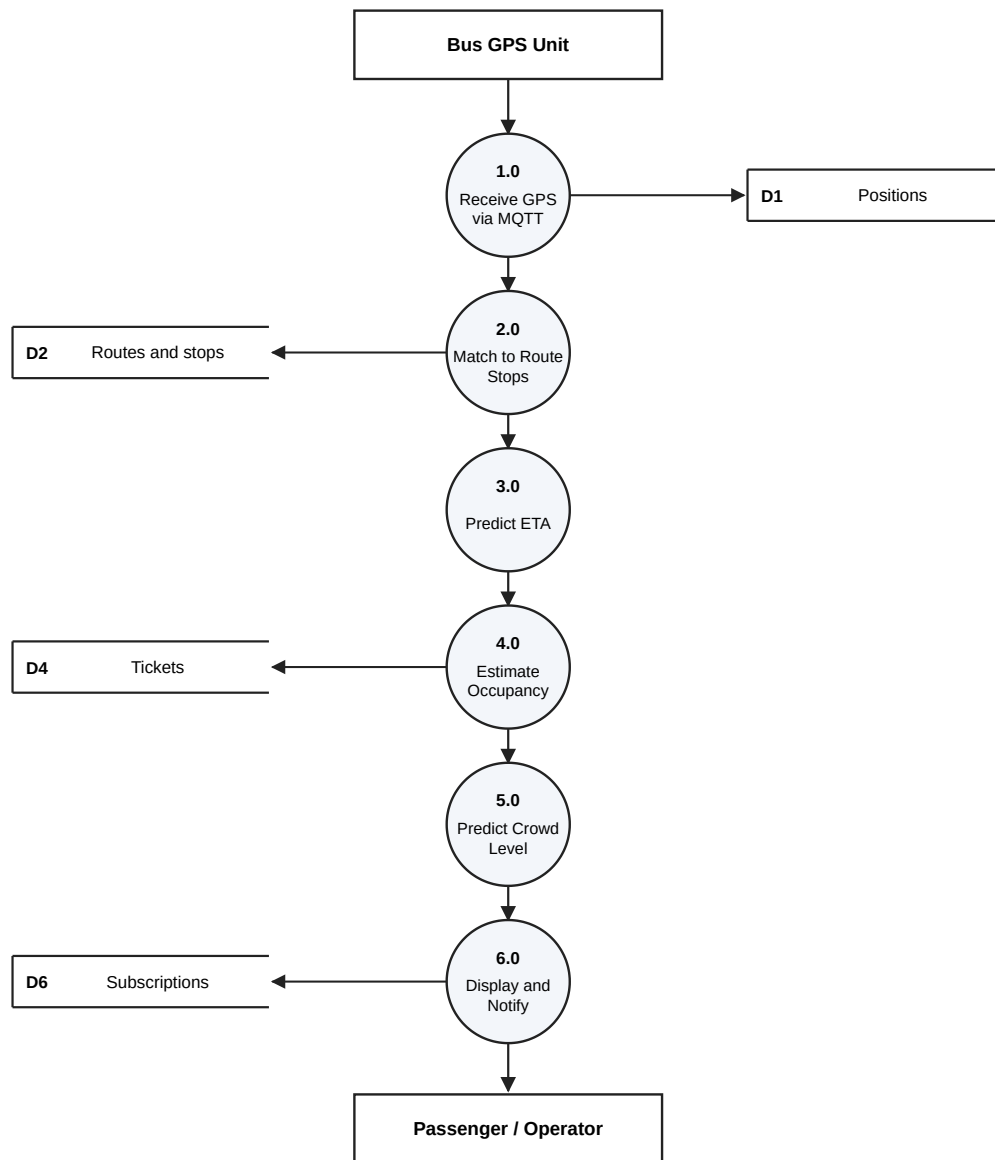


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

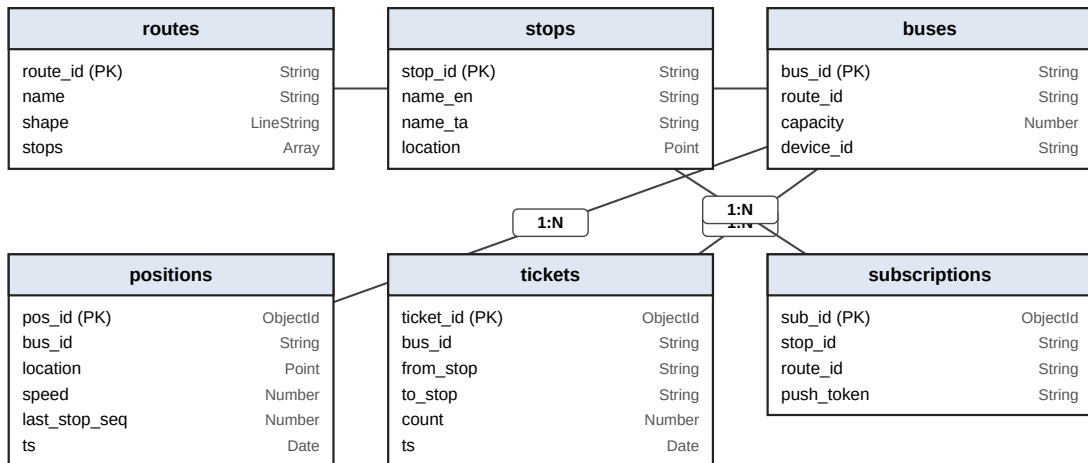


Fig 5: Entity relationship diagram

6.5 Table Design

ROUTES

FIELD NAME	DATA TYPE	DESCRIPTION
route_id	String	Primary key (e.g. 12A)
name	String	Route name
shape	LineString	Route geometry
stops	Array	Ordered stop ids

STOPS

FIELD NAME	DATA TYPE	DESCRIPTION
stop_id	String	Primary key
name_en	String	Stop name
name_ta	String	Stop name in Tamil
location	Point	Coordinates (2dsphere index)

BUSES

FIELD NAME	DATA TYPE	DESCRIPTION
bus_id	String	Primary key (registration)
route_id	String	Foreign key to routes

capacity	Number	Seats + standing capacity
device_id	String	GPS unit id

POSITIONS

FIELD NAME	DATA TYPE	DESCRIPTION
pos_id	ObjectId	Primary key
bus_id	String	Foreign key to buses
location	Point	GPS point
speed	Number	km/h
last_stop_seq	Number	Last passed stop
ts	Date	Time

TICKETS

FIELD NAME	DATA TYPE	DESCRIPTION
ticket_id	ObjectId	Primary key
bus_id	String	Foreign key to buses
from_stop	String	Boarding stop
to_stop	String	Alighting stop
count	Number	Passengers
ts	Date	Time

SUBSCRIPTIONS

FIELD NAME	DATA TYPE	DESCRIPTION
sub_id	ObjectId	Primary key
stop_id	String	Foreign key to stops
route_id	String	Route
push_token	String	Web push token

6.6 Algorithm Design

ETA and crowd prediction are computed every time a new GPS point arrives for a bus.

1. **Bus device sends GPS every few seconds** – NMEA sentences are parsed with TinyGPS++ and published as JSON every 5 seconds.
2. **Match location to route stops** – The point is projected onto the route polyline; the distance along the route determines the last passed stop.
3. **Predict ETA using historical speeds and live traffic** – For each upcoming segment, a LightGBM model predicts travel time from hour, weekday, historical median speed, current bus speed and speeds of buses that passed the segment in the last 15 minutes; dwell time is added per stop.
4. **Estimate occupancy from ticket counts or sensors** – Load = boarded - alighted (from tickets with destination stop), or from IR door counters.
5. **Predict crowd level using regression model** – Predicted load at stop k = current load + predicted boardings - predicted alightings for intermediate stops; mapped to Seats (<70%), Standing (70-100%), Full (>100%).
6. **Show live bus on map with ETA** – Leaflet map with bus markers updated through WebSocket.
7. **Notify subscribed passengers** – Web push when ETA to the subscribed stop is 5 minutes or less.
8. **Generate route performance reports** – Daily aggregation of punctuality, running times, headway regularity and demand.

PSEUDO CODE

```
ON POSITION(bus, p):
  s <- PROJECT_ON_ROUTE(p, route.shape)           // distance along route
  k <- LAST_STOP_PASSED(s, route.stop_distances)
  t <- now
  FOR j in k+1 .. last_stop:
    seg_time <- ETA_MODEL(seg(j-1, j), hour(t), weekday, hist_speed,
bus.speed, recent_speed(seg))
    t <- t + seg_time + DWELL(j)
    eta[j] <- t
    load <- load + PRED_BOARD(j, hour) - PRED_ALIGHT(j, hour)
    crowd[j] <- LEVEL(load / bus.capacity)
```

```
BROADCAST(bus, p, eta, crowd)
NOTIFY subscribers WHERE eta[stop] - now <= 5 min
```

6.7 Modules

The modules are grouped by the Passenger, Conductor and Operator users.

MODULES

PASSENGER

1. SEARCH STOP OR ROUTE
2. VIEW LIVE BUSES AND ETA
3. VIEW CROWD LEVEL
4. SUBSCRIBE TO ARRIVAL ALERT

CONDUCTOR

1. RECORD TICKETS

OPERATOR

1. MONITOR FLEET
2. MANAGE ROUTES AND STOPS
3. VIEW REPORTS

PASSENGER

SEARCH STOP OR ROUTE

Passengers search for a stop or route number in Tamil or English, or use the nearest stop detected from GPS.

VIEW LIVE BUSES AND ETA

The map shows buses on the route with their predicted arrival times at the selected stop.

VIEW CROWD LEVEL

Each upcoming bus shows the predicted crowd level at the passenger's stop: Seats available, Standing or Full.

SUBSCRIBE TO ARRIVAL ALERT

Passengers get a notification when the selected bus is 5 minutes away.

CONDUCTOR

RECORD TICKETS

The conductor app or electronic ticket machine records tickets issued per stop, which updates the bus load.

OPERATOR

MONITOR FLEET

Operators see all buses, delays and bunching alerts on a map.

MANAGE ROUTES AND STOPS

Operators define routes, stops and schedules.

VIEW REPORTS

Punctuality, running time variability, stop-wise demand and crowding reports.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

The BusNow tracking unit firmware was written in C++ using the Arduino IDE with the TinyGPS++ and PubSubClient libraries. A Node.js service subscribes to MQTT, performs map matching using Turf.js, stores positions in MongoDB and broadcasts updates via Socket.IO. A Python service hosts the LightGBM ETA and crowd models.

7.2 Implementation

The system was piloted on two town bus routes for three weeks using four buses. Historical data from the first two weeks was used for training and the third week for testing. Conductor ticket counts were entered through a simple Android app with stop buttons.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	GPS publishing	Bus unit powered	Position on map within 10 s	Pass
TC02	Map matching	Bus at stop 'Old Bus Stand'	Last stop updated correctly	Pass
TC03	ETA prediction	Bus 3 stops away	ETA within 2 min of actual	Pass
TC04	Network loss	No signal for 3 min	Buffered points uploaded later	Pass
TC05	Crowd level	Load 58 / capacity 60	Shown 'Standing'	Pass
TC06	Arrival alert	Subscribed to stop	Notification 5 min before	Pass
TC07	Bunching detection	Two buses within 2 min	Operator alert	Pass
TC08	Report	Route 12A, last week	Punctuality and demand report	Pass

PERFORMANCE EVALUATION

ETA accuracy on the test week:

Method	MAE (min)	% within 2 min
Timetable	6.8	31%
Distance / current speed	3.4	55%
Historical average	2.6	64%
LightGBM (proposed)	1.4	83%

7.4 Source Code

bus_unit.ino (ESP32 + NEO-6M)

```
#include <TinyGPS++.h>
#include <HardwareSerial.h>
```

```

#include <WiFi.h>
#include <PubSubClient.h>

TinyGPSPlus gps;
HardwareSerial gpsSerial(2);          // RX=16, TX=17
WiFiClient net; PubSubClient mqtt(net);
const char* BUS = "TN49N1234";
String buffer[60]; int buffered = 0;   // keep points during network loss

void setup() {
  gpsSerial.begin(9600, SERIAL_8N1, 16, 17);
  WiFi.begin("BUS_HOTSPOT", "password"); // 4G router / phone hotspot on bus
  mqtt.setServer("mqtt.busnow.in", 1883);
}

void loop() {
  while (gpsSerial.available()) gps.encode(gpsSerial.read());
  static unsigned long last = 0;
  if (millis() - last > 5000 && gps.location.isValid()) {
    last = millis();
    char msg[160];
    snprintf(msg, sizeof msg, "
{\\bus\\:\\\"s\\\",\\\"lat\\\":%.6f,\\\"lng\\\":%.6f,\\\"spd\\\":%.1f,\\\"hdg\\\":%.0f,\\\"t\\\":%lu}\\\",
      BUS, gps.location.lat(), gps.location.lng(), gps.speed.kmph(),
gps.course.deg(), gps.time.value());
    if (WiFi.status() == WL_CONNECTED && (mqtt.connected() ||
mqtt.connect(BUS))) {
      for (int i = 0; i < buffered; i++) mqtt.publish("busnow/pos",
buffer[i].c_str());
      buffered = 0;
      mqtt.publish("busnow/pos", msg);
    } else if (buffered < 60) {
      buffer[buffered++] = String(msg);
    }
  }
  mqtt.loop();
}

```

ingest.js (Node.js map matching)

```

const mqtt = require("mqtt");
const turf = require("@turf/turf");
const axios = require("axios");
const { MongoClient } = require("mongodb");
const io = require("socket.io")(5002, { cors: { origin: "*" } });

(async () => {
  const db = (await
MongoClient.connect("mongodb://localhost:27017")).db("busnow");
  const routes = {};
  for (const r of await db.collection("routes").find().toArray()) {
    const line = turf.lineString(r.shape.coordinates);
    const stops = await db.collection("stops").find({ stop_id: { $in: r.stops }
}).toArray();
    r.stopDist = r.stops.map(id => {
      const s = stops.find(x => x.stop_id === id);
      return { id, name: s.name_en, d: turf.nearestPointOnLine(line,
s.location.coordinates, { units: "meters" }).properties.location };
    });
    routes[r.route_id] = { ...r, line };
  }
}

```

```

const client = mqtt.connect("mqtt://localhost");
client.on("connect", () => client.subscribe("busnow/pos"));
client.on("message", async (_, payload) => {
  const p = JSON.parse(payload);
  const bus = await db.collection("buses").findOne({ bus_id: p.bus });
  const route = routes[bus.route_id];
  const along = turf.nearestPointOnLine(route.line, [p.lng, p.lat], { units:
"meters" }).properties.location;
  const lastSeq = route.stopDist.filter(s => s.d <= along + 30).length - 1;
  await db.collection("positions").insertOne({ bus_id: p.bus, location: {
type: "Point", coordinates: [p.lng, p.lat] },
                                speed: p.spd, along,
last_stop_seq: lastSeq, ts: new Date() });
  const { data } = await axios.post("http://localhost:8001/predict", {
    bus_id: p.bus, route_id: bus.route_id, along, speed: p.spd, last_stop_seq:
lastSeq,
    upcoming: route.stopDist.slice(lastSeq + 1), capacity: bus.capacity });
  io.to(bus.route_id).emit("bus", { bus: p.bus, lat: p.lat, lng: p.lng, eta:
data.eta, crowd: data.crowd });
});
})();

```

predict_service.py (ETA and crowd)

```

import datetime as dt, joblib, numpy as np
from fastapi import FastAPI
from pymongo import MongoClient

app = FastAPI()
eta_model = joblib.load("models/segment_time_lgbm.pkl")
board = joblib.load("models/boardings_by_stop_hour.pkl") # dict[(stop,
hour)] -> (board, alight)
db = MongoClient()["busnow"]

def current_load(bus_id):
    today = dt.datetime.combine(dt.date.today(), dt.time())
    b = sum(t["count"] for t in db.tickets.find({"bus_id": bus_id, "ts":
{"$gte": today}}))
    a = sum(t["count"] for t in db.tickets.find({"bus_id": bus_id, "ts":
{"$gte": today}, "alighted": True}))
    return max(b - a, 0)

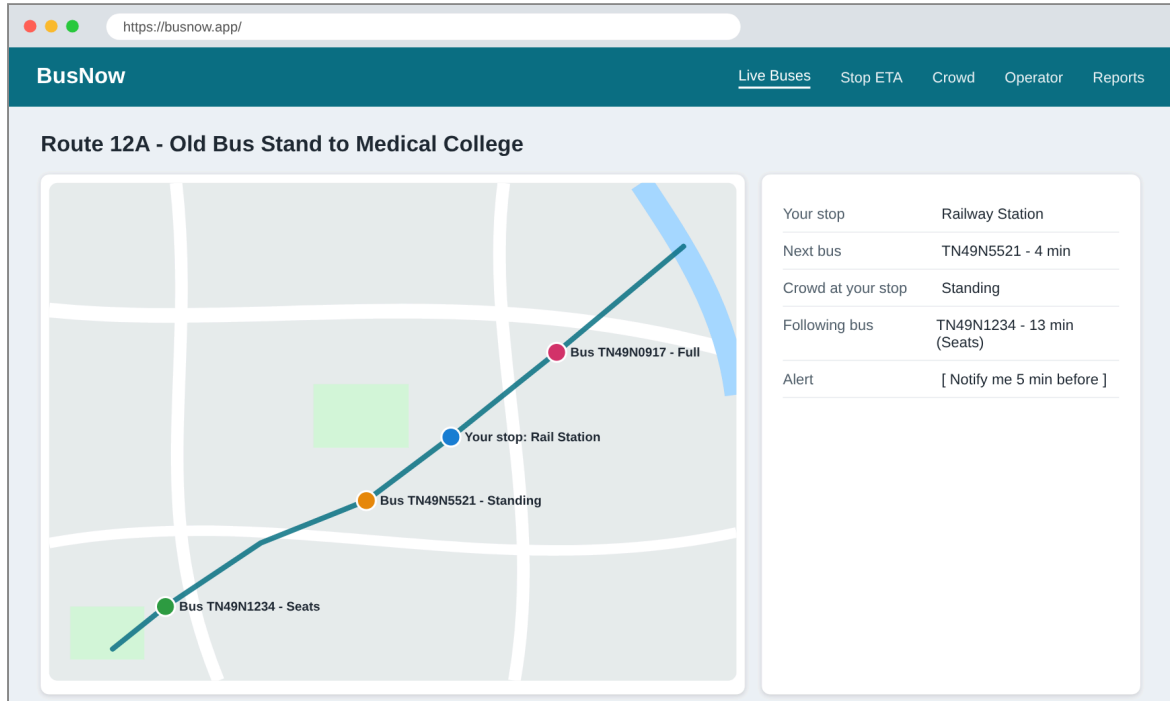
@app.post("/predict")
def predict(req: dict):
    now = dt.datetime.now(); t = now; prev = req["along"]
    load = current_load(req["bus_id"]); eta, crowd = {}, {}
    for s in req["upcoming"]:
        dist = s["d"] - prev
        X = np.array([[now.hour, now.weekday(), dist, req["speed"]]])
        t += dt.timedelta(seconds=float(eta_model.predict(X)[0]) + 20) # +
dwell time
        bo, al = board.get((s["id"], t.hour), (2, 2))
        load = max(load + bo - al, 0)
        ratio = load / req["capacity"]
        eta[s["id"]] = t.strftime("%H:%M")
        crowd[s["id"]] = "Seats" if ratio < 0.7 else "Standing" if ratio <= 1.0
    else "Full"
    prev = s["d"]
    return {"eta": eta, "crowd": crowd}

```

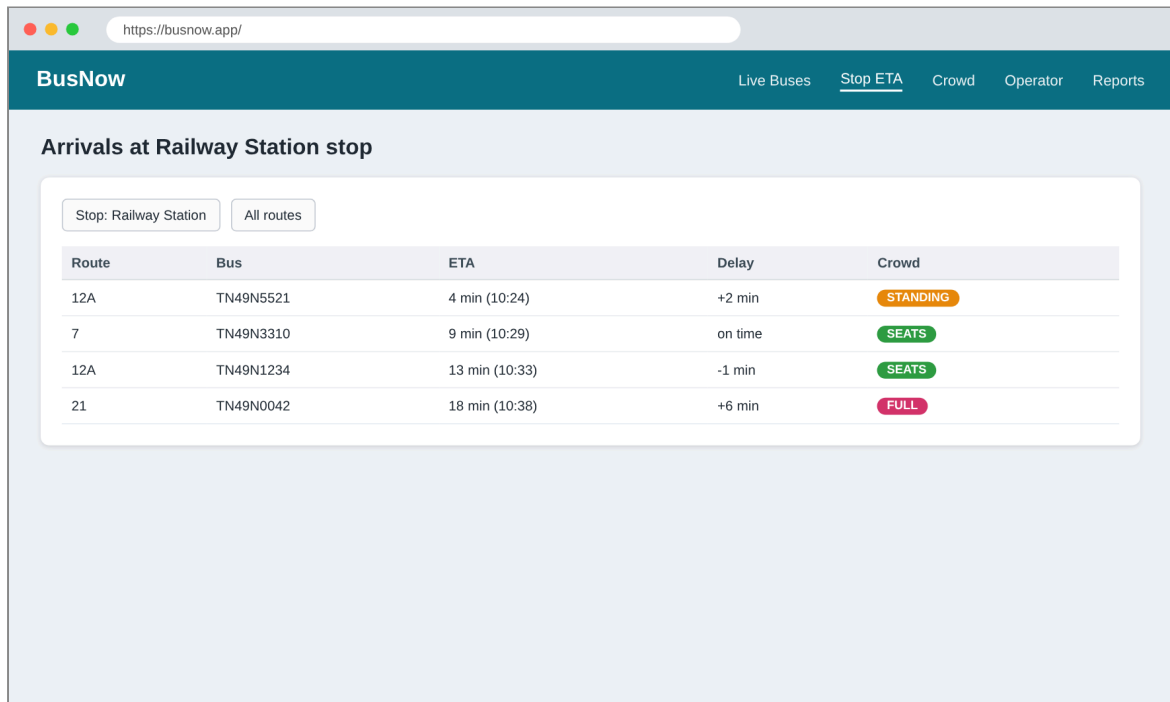
7.5 Screen Layout

SCREENSHOTS:

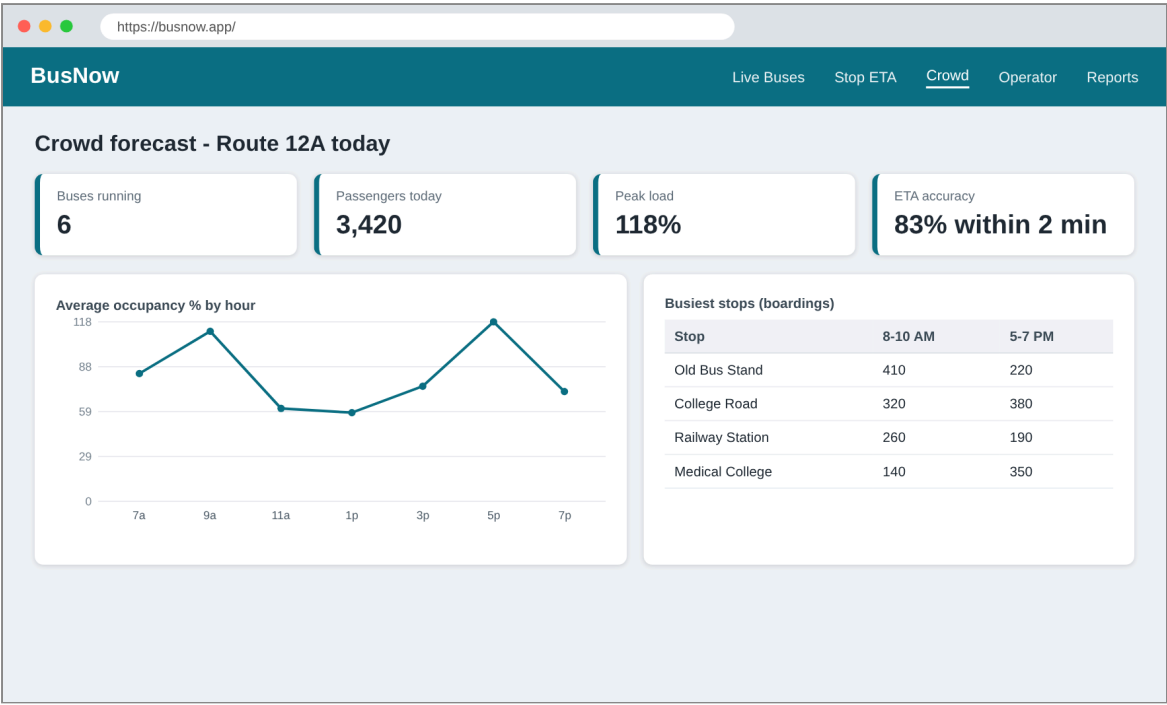
LIVE BUSES ON ROUTE



STOP ETA BOARD



CROWD PREDICTION



CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Smart Bus Tracking and Crowd Prediction system provides passengers with live bus locations, accurate arrival times and crowd levels using a low-cost ESP32 GPS unit and machine learning. The LightGBM ETA model achieved a mean absolute error of 1.4 minutes, with 83% of predictions within 2 minutes, far better than timetable-based estimates.

Crowd predictions help passengers choose less crowded buses, and operator reports highlight peak overloads and delays for schedule adjustments. The system can make public transport more reliable and attractive, especially on town and rural routes that currently lack any real-time information.

8.2 Future Enhancement

- GTFS-Realtime feed publication for Google Maps integration.
- Camera-based passenger counting with on-device AI.
- SMS-based ETA queries for feature phones.
- Dynamic scheduling and headway control.
- Integration with digital ticketing and NCMC cards.

CHAPTER IX

BIBLIOGRAPHY

1. Jeong, R., and Rilett, L. R., "Bus arrival time prediction using artificial neural network model", Proc. IEEE Intelligent Transportation Systems Conference, 2004.
2. Kalman, R. E., "A New Approach to Linear Filtering and Prediction Problems", Journal of Basic Engineering, Vol. 82, pp. 35-45, 1960.
3. Friedman, J. H., "Greedy Function Approximation: A Gradient Boosting Machine", Annals of Statistics, Vol. 29, No. 5, pp. 1189-1232, 2001.
4. Ke, G. et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", Advances in Neural Information Processing Systems (NeurIPS), 2017.
5. Jenelius, E., "Data-Driven Metro Train Crowding Prediction Based on Real-Time Load Data", IEEE Transactions on Intelligent Transportation Systems, Vol. 21, No. 6, 2020.
6. Google, "GTFS Realtime Reference", <https://gtfs.org/realtime>.

WEBSITE REFERENCE

1. <https://gtfs.org>
2. <https://mosquitto.org>
3. <https://turfjs.org>
4. <https://leafletjs.com>